

Jour 1 — Étage régulier

Automates finis & transducteurs

Modules : `formlang/dfa.py`, `nfa.py`, `fst.py` • Tests : `test_regular.py` • Barème : 4 pts (+1)

Cadre éthique

Alphabet abstrait $\{a, o, r\}$ (+ chiffres *leet*). Aucun contenu réel.

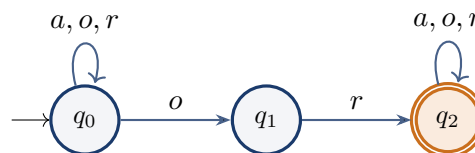
Contexte

Le premier réflexe d'un détecteur est de repérer un **motif** : « le mot contient le facteur **or** » — un langage **régulier**. Avant de détecter, on **normalise** (défaire le *leetspeak* $4 \rightarrow a, 0 \rightarrow o, \dots$) : une relation **rationnelle** réalisée par un **transducteur fini**. Un mot $a_1 \dots a_n$ est un arbre dégénéré (branche unaire) ; l'automate fini est le cas « tout en arité 1 » du jour 3.

Notions nouvelles du jour

1. AFD et AFN

Un **AFD** lit un mot de gauche à droite ; dans chaque état, chaque lettre mène à *un seul* état. Un **AFN** autorise *plusieurs* transitions pour une même lettre : on accepte s'il *existe* un chemin acceptant. L'AFN pour « contient **or** » est très naturel — on « devine » où commence le facteur :



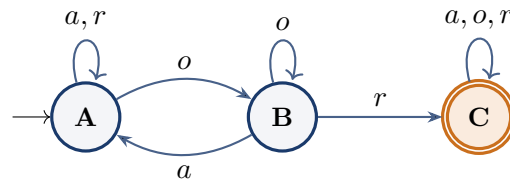
Le **non-déterminisme** est visible en q_0 : sur **o**, on peut *rester* en q_0 **ou** *aller* en q_1 .

2. Déterminisation par construction des sous-ensembles

Un état de l'AFD = un *ensemble* d'états de l'AFN (ceux où « on pourrait être »). On part de $\{q_0\}$ et on ferme par transition :

état AFD	sur o	sur a,r	remarque
$S_0 = \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$	initial
$S_1 = \{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0\}$ (sur a), $\{q_0, q_2\}$ (sur r)	o « armé »
$S_2 = \{q_0, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_2\}$	accepteur (q_2)

Après renommage et fusion des états équivalents (minimisation), on retrouve **trois** états — l'AFD minimal ci-dessous.



Trace de l'AFD sur aor

pas	état	lit	transition
0	A	a	$A \xrightarrow{a} A$
1	A	o	$A \xrightarrow{o} B$
2	B	r	$B \xrightarrow{r} C$
3	C	—	C final $\Rightarrow$ accepté

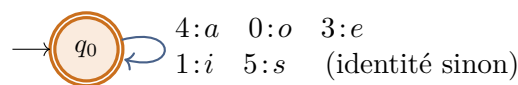
En B, on est « armé » : un **r** immédiat fait basculer en C (absorbant). **aoa** au contraire retombe de B en A : rejeté.

Minimisation (Moore)

Deux états sont **indistinguables** s'ils acceptent exactement les mêmes suffixes. La minimisation **fusionne** ces états : partition initiale finaux/non-finaux, puis **raffinement** tant que deux états d'un bloc mènent à des blocs différents. Le résultat (l'AFD minimal) est **unique**. • Code : `DFA.minimize`; lien direct avec Myhill–Nerode (jour 5).

3. Transducteur fini (FST) : normaliser le *leet*

Un AFD *reconnaît* ; un **transducteur** *traduit* : chaque transition *émet* une sortie ($x:y$ = « lit x , écrit y »). Le normaliseur *leet* tient en **un seul état** (identité sur les lettres non listées) :



Transduction et idempotence

$T(4or) = aor$; $T(r0le) = role$. La sortie ne contient **plus** de chiffre *leet*, donc $T(T(w)) = T(w)$: le transducteur est **idempotent** (test `test_leet_idempotent`). La **composition** $T_2 \circ T_1$ se lit sur les paires d'états (fonction `compose`).

4. Une limite déjà visible : one-way vs two-way

Le renversement $w \mapsto w^R$ est **impossible** avec un transducteur **one-way** : il faudrait mémoriser tout w avant d'émettre la première lettre de sortie — impossible à mémoire bornée (nombre fini d'états). Une tête **two-way** (qui revient en arrière) le permet. Premier exemple concret de *limite due à la mémoire* — le fil rouge du projet, qui culminera au jour 4.

A. Théorie (à rédiger) — 2 pts

On pose $\Sigma = \{a, o, r\}$ et $L_1 = \{w \mid w \text{ contient le facteur } or\}$.

Q1.1 (0,5)

Donner une expression régulière dénotant L_1 .

Q1.2 (1)

AFN pour L_1 , **déterminiser** (table des sous-ensembles) puis **minimiser** ; donner la table de l'AFD minimal, le dessiner, indiquer le nombre d'états.

Q1.3 (0,5)

$L'_1 = \{w \mid w \text{ contient un } o \text{ suivi, pas forcément immédiatement, d'un } r\}$: regex, différence avec L_1 , inclusion $L_1 \subseteq L'_1$.

B. Pratique — 2 pts

E1.1 (0,75)

`contains_or sous forme d'AFD` (table δ , pas l'opérateur `in`) + `DFA.run/accepts`.
Test : test_detector_or.

E1.2 (0,5)

`DFA.minimize`. *Test : test_minimisation_3_etats.*

E1.3 (0,5)

`NFA.to_dfa`. *Test : test_nfa_to_dfa_equivalence.*

E1.4 (0,25)

`leet_fst + SequentialFST.transduce`. *Tests : test_leet_idempotent, test_fst_composition.*

Sortie de référence

```
$ pytest -q tests/test_regular.py
7 passed
```

Mini-barème

Item	Détail	Pts
Q1.1–Q1.3	regex, AFN $\rightarrow$ AFD $\rightarrow$ min, facteur vs sous-séquence	2
E1.1	détecteur AFD (<code>contains_or</code>)	0,75
E1.2	minimisation 3 états	0,5
E1.3	déterminisation	0,5
E1.4	FST leet + composition	0,25
Bonus	construction de Thompson (regex $\rightarrow$ AFN)	+1

Gate (règle non négociable)

`detector.py` **instancie** `formlang.dfa.DFA` ; il ne teste pas "or" in w.

Références

M. Sipser, *Introduction to the Theory of Computation*, 3^e éd., 2012. • J. Hopcroft, R. Motwani, J. Ullman, *Introduction to Automata Theory...*, 3^e éd., 2006.